

GitHub - bharathganeshsivasankaran/Assignment3

bharathganeshsivasankaran

GPT vs. Claude: Coding Task Comparison

1. Coding Task

For this comparison, I gave the same coding question to both GPT and Claude. I kept the prompt the same because I wanted to see how both models handled the same requirement.

Task: Create a Python function called `calculate_average()` that takes a list of numbers and returns their average. It should also handle an empty list without giving an error.

Prompt given to both models

Write a simple Python function called `calculate_average(numbers)`. It should calculate and return the average of the numbers in the list. If the list is empty, return 0. Also provide two simple test cases and briefly explain the code.

I did not change the requirement between the two models.

2. GPT Response

GPT gave a simple solution using `sum()` and `len()`. It first checks whether the list is empty and returns 0 if there are no values.

```
def calculate_average(numbers):  
    if not numbers:  
        return 0  
    return sum(numbers) / len(numbers)
```

I found this solution easy to understand. The empty-list condition is also useful because otherwise the program could cause a division-by-zero error. The test cases were simple and could be checked easily.

3. Claude Response

Claude also gave a simple solution for the same problem.

```
def calculate_average(numbers):  
    if len(numbers) == 0:  
        return 0  
    return sum(numbers) / len(numbers)
```

The logic is basically the same as the GPT solution. Claude used `len(numbers) == 0` to check the empty list, while GPT used `if not numbers`. Both approaches work for this requirement.

The explanation was also clear and the code was easy to follow.

4. Comparison

Factor	GPT	Claude
Correctness	Correct	Correct
Code complexity	Simple	Simple
Readability	Easy to understand	Easy to understand
Empty-list handling	Yes	Yes
Explanation	Short and direct	Clear and detailed
Overall	Good for quick coding	Good for understanding the solution

For this particular example, I did not find a major difference in the actual coding logic. Both models produced working solutions and handled the empty-list case correctly. Since the task was simple, both models were able to solve it without much difficulty.

My Observation

From this comparison, I understood that an AI coding tool should not be judged only by whether it gives code that works. I still need to read the code and check whether it actually matches the requirement and handles possible edge cases.

I also noticed that two different AI models can give slightly different implementations for the same problem while still producing the correct result. So, instead of directly copying the generated code, I would review it and then use the version that fits my project better.

Data Privacy Checklist for Using Third-Party LLMs with Guest PII

In our hospitality project, we may deal with guest information. Because of this, I would be careful about what information is given to a third-party AI tool.

1. Avoid sending unnecessary guest information

I would not send information such as a guest's full name, phone number, email address, room number, passport details, or payment information unless it is genuinely required for the task.

2. Anonymize guest information

If some guest information is needed for the AI to understand a problem, I would replace personal details with placeholders.

For example:

Instead of:

Guest Arun Kumar, phone number 9876543210, staying in Room 204, complained about the service.

I would use:

Guest [GUEST_ID_102], staying in [ROOM], complained about the service.

This gives the AI the information it needs about the problem without directly exposing the guest's identity.

3. Do not share passwords or credentials

I would never paste production passwords, API keys, database credentials, authentication tokens, or payment-card information into an external AI tool.

Even if the AI provider says that the information is not used for training, the information has still been sent to an external service, so there is still a security concern.

4. Check the AI provider's privacy policy

Before using a third-party LLM with company or guest information, I would check its current policies regarding data storage, retention, training, and processing.

One important point from this exercise is that a company saying "we do not train our model using your data" does not automatically mean that we should send any type of data to that company.

5. Send only the information that is needed

I would follow a minimum-data approach. For example, if the AI only needs to understand a guest's food preference, there is no need to send the guest's phone number, payment information, or other unrelated details.

6. Keep human review for sensitive situations

I would not allow an LLM to make important decisions about guests completely on its own. For sensitive requests or decisions, a staff member should review the AI's response before taking action.

Final Checklist

- Remove unnecessary guest PII.
- Replace personal information with anonymous IDs or placeholders.
- Never share passwords, API keys, or database credentials.
- Never share payment-card information.
- Check the third-party LLM's privacy and data-retention policies.
- Send only the minimum information required.
- Have a human review sensitive AI-generated responses.
- Remember that "not used for training" does not automatically mean "safe to send."

Conclusion

This exercise made me realize that LLMs can be useful for coding and other tasks, but we should not blindly trust everything they generate. In a hospitality application, privacy is even more important because guest information can be sensitive.

My approach would be to use AI to save development time while being careful about the data I provide to it. I would remove unnecessary personal information, use anonymized data wherever possible, and keep human review for important decisions.